

Review of BinaryPart 1: Binary Numbers

- an N -bit unsigned number can represent values from 0 up to $2^N - 1$, so $0, 1, 2, 3, \dots, 2^N - 1$

2^N things in total

- binary to decimal: $(101)_b \rightarrow 1(2)^2 + 0(2)^1 + 1(2)^0 = (5)_d$
- decimal to binary: can do the whole division thing or write out enough binary-decimal pairs as dual list and "build" your decimal using highest possible binaries.
- hex digit is a 4-bit binary, which you can convert to decimal.
- signed numbers: sign + magnitude, 1's complement, 2's complement

use MSB as sign

bit: 0 - (+)ve
1 - (-)ve

eg: 3 - 011

-3 - 111

rest of bits stay

the same for
the number

MSB still indicates the sign. If it's (+)ve, take number as-is, but if it's negative, must take complement of the ~~rest of the~~ bits to know what number's (-)ve version you have at hand.

likewise, MSB still indicates sign and for (+)ve ones, take value as-is; for (-)ve values, must take complement of the bits and add 1 to see what number's (-)ve version you have

- N-bit unsigned $\rightarrow$ represent values $[0, 2^N - 1]$
- N-bit signed $\rightarrow$ represent values $[-2^{N-1}, 2^{N-1} - 1]$
- sign extension: copy over MSB to every higher order bit

Part 2: Binary Arithmetic

- Addition and Subtraction

$$\begin{array}{r} \swarrow \\ 1101 \\ + 110 \\ \hline 10011 \end{array}$$

$$\begin{array}{r} 01010 \\ 1101 \\ - 110 \\ \hline 0110 \end{array}$$

- for subtracting with 2's complement, "add the negative":

$$13 - 6 \rightarrow 13 + -6$$

$$\begin{array}{cc} \swarrow & \downarrow \\ 01101 & 11010 \end{array}$$

$$\begin{array}{r} 01101 \\ + 11010 \\ \hline 100111 \end{array}$$

↑
discard. answer in 2's complement

- always sign-extend numbers
b4 adding them

Part 3: Binary in C

uint8_t a = 0b1101;

0xFF2;

← binary

← hex

%u, %d, %x assume
32-bit inputs

%X is hex value
of a char

Fixed Point and Floating Point

Feb 1, 2026

Fixed Point

32 bits → Qm.n: signed, m integer, n fractional
 UQm.n: unsigned, —//—//—

for 32 bits, eg: Q31.0, Q29.2, Q2.29, UQ32.0, UQ2.30

• $(6.625)_d = (110\ 101)_{UQ3-3}$
 $6 = 110 = 2^2 + 2^1 = 4 + 2 = 6$

$0.625 \times 2 = 1.25$
 $0.25 \times 2 = 0.5$
 $0.5 \times 2 = 1.0$
 $0 \times 2 = 0$

* for changing formats,
 if larger format, sign
 extend MSB for integer
 bits and "zero extend"
 LSB for frac-bits.

• Resolution: $\leftarrow$ min gap b/n numbers: For both: 2^{-n} Qm.n
 • resolution ↑, range ↓ (if (-)ve, convert 2's compl. and $\div 2^n$)

max - min
 eg: $\frac{127}{2^n} - \frac{-128}{2^n}$
 for Q5.2

Addition

* adding 2 k-bit
 numbers results in
 (k+1) bit answer
 * line up decimals
 and add/sub
 as needed

Mult.

UQa.b
 $\times$ UQc.d
 UQ(a+b).(b+d)
 * bits
 $3.25 \times 2.75 = 8.9375$
 $325 \times 275 = 89375$

Div.

case 2: $1.5 \div 0.25 = 6$
 $110 \div 001 = 0110$
 UQ2.2, but int
 if diff format, match
 to highest frac-bits.

Floating Point

$\boxed{\text{sign}} \mid 1. \boxed{\text{fraction (mantissa)}} \times 2^{\boxed{\text{exponent}}}$
 IEEE 754 format: single precision: $\boxed{1} \mid \boxed{8 \text{ bits}} \mid \boxed{23 \text{ bits}}$
 sign exponent mantissa

double precision: $\boxed{1} \mid \boxed{11 \text{ bits}} \mid \boxed{52 \text{ bits}}$
 sign exponent mantissa (fraction)

• In 32 bit FP: exponent = actual exponent + 127
 (double, add 1023, half, add 15)

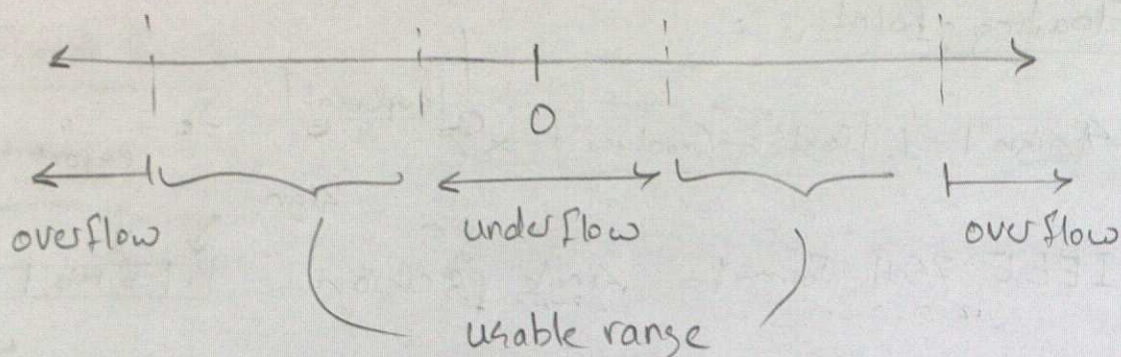
F: 127
 d: 1023

Formatted
 real number: $(-1)^{\text{sign}} \times (1 + \text{mantissa}) \times 2^{(\text{exponent} - \text{bias})}$
 stored as bit sign stored as UQ0.23 for single precision (convert to decimal bit adding +1) stored as regular binary. (aka the biased exponent, meaning the actual exp. with the bias number added.)

Conversion:

- sign
- normalize (get 1.something)
- mantissa
- exponent

sign	exponent	fractional	meaning
0/1	00...0	00...0	± ZERO
0/1	11...1	00...0	± ∞
0/1	00...0	any non-zero	subnormal
0/1	11...1	any non-zero	NaN



• resolution: gaps get larger as numbers get bigger

Multiplication

$$\begin{array}{|c|c|c|} \hline S_1 & E_1 & M_1 \\ \hline \end{array}
 \begin{array}{c} * \\ \\ = \end{array}
 \begin{array}{|c|c|c|} \hline S_2 & E_2 & M_2 \\ \hline \end{array}
 =
 \begin{array}{|c|c|c|} \hline S_R & E_R & M_R \\ \hline \end{array}$$

- 1) sign bit: $S_1 \oplus S_2$
- 2) multiply mantissas and normalize if needed: $M_1 * M_2$
- 3) add exponents (and normalization adjustment if needed): $E_1 + E_2$
- 4) check for overflow and store values as needed

Addition

$$\begin{array}{|c|c|c|} \hline S_1 & E_1 & M_1 \\ \hline \end{array}
 \begin{array}{c} + \\ \\ = \end{array}
 \begin{array}{|c|c|c|} \hline S_2 & E_2 & M_2 \\ \hline \end{array}
 =
 \begin{array}{|c|c|c|} \hline S_R & E_R & M_R \\ \hline \end{array}$$

- 1) match exponents if needed (match the smaller)
- 2) add mantissas
- 3) normalize mantissa
- 4) check for overflow and save specified values.

Basic Peripherals

* GPIO: tristate buffer

- pullup, pull down
- floating
- open-drain: sinking current
- push-pull: sourcing current

* Timers (SysTick)

- prescaler value K divides clock frequency by K (multiplies clock period by K)
- (tradeoffs b/w precision and range)

* Analog Comparators

* ADC's

- 2 stages: 1) sample and hold
2) quantization
- Flash ADC (lots of comparators)
(big voltage divider feeding comparators)

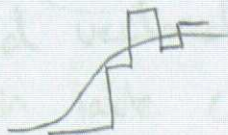
sampling rate
error is affected by

* bits
(resolution)

fast, but expensive

- Successive Approx ADC (sample and hold $\rightarrow$ amplifier $\rightarrow$ comparator)
(n steps for n bits)

more resolution means slower



DAC's/Hubs

- Binary Weighted DAC (weighted sum amplifier)

PWM

- another way to produce analog signal

Memory Regions

* setting one bit: $\text{num} \mid= (1 \ll x);$

* clearing one bit: $\text{num} \&= \sim(1 \ll x);$

* toggling one bit: $\text{num} \wedge= (1 \ll x);$

Keywords:

- volatile: use as memory address only

- extern: for global variables declared in another file.

- static: initialized only once and retains its value (persists) across funⁿ calls

- const: "read-only" variable (cannot change).

RAM

fast
small
volatile

ROM

slow
big
non-volatile

volatile int var1;

int var2 = 3;

const int var3 = 10;

- var1 in RAM

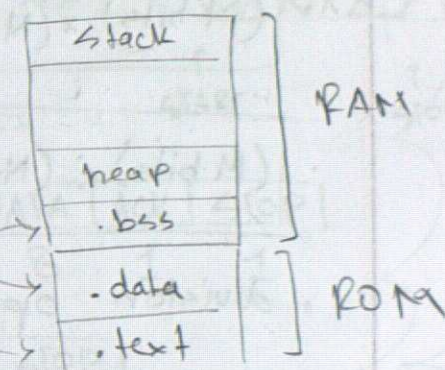
- var2 in ROM, copied to RAM on boot

- var3 always in ROM

.text: prog code

.data: initialized data

.bss: uninitialized data



Interrupts

* can disable/mask interrupts for critical secⁿs

* In 243, had one interrupt request handler (whose address we had to load into the CPU prior to starting) inside which we checked which interrupt happened before calling/jumping to specific ISR. * 3 interrupt priorities

* In 342, they called the 243 as having one massive ISR

* 342, your processor already knows addresses of specific ISR's from table in Vector Interrupt Controller

* Nested vectored int. contr: several interrupt vecs per main table entry (must check to know which specific one)

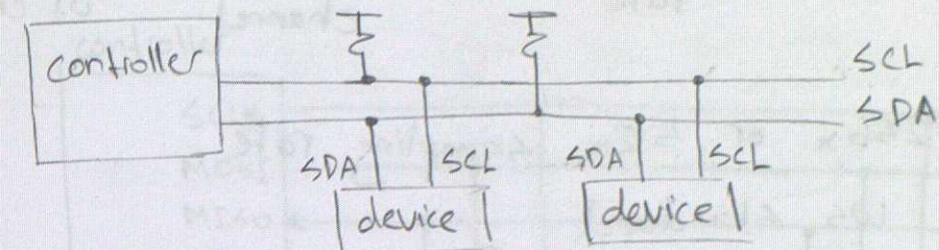
Binary Arithmetic

- N-bit unsigned: $[0, 2^N - 1]$
- N-bit signed: $[-2^{N-1}, 2^{N-1} - 1]$ $\rightarrow (N-1)$ bc one bit for sign
- sign extend signed nums. $\rightarrow 2^{N-1} - 1$ for (+)ve since one place for 0 already.
- $(M \text{ bits}) \times (N \text{ bits}) = (M+N) \text{ bits}$
- $(M \text{ bits}) \div (N \text{ bits}) = (M-N+1) \text{ bits and } R(N) \text{ bits}$
- division optimized by mults. and shifts

Bus

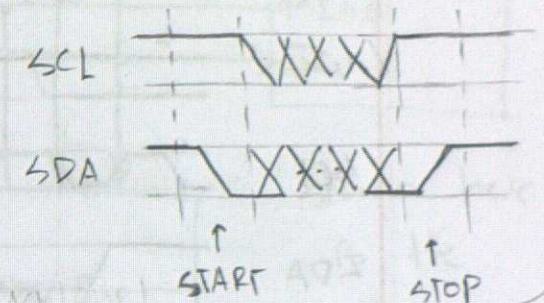
throughput = (datawidth * bits) - overhead

I2C : inter-integrated circuit



- half-duplex
- needs arbitration
- MSB first

- 3 clock stretching
- posedge clk



writing:

START	ID	R/W	ACK	ADDR	ACK	DATA	ACK	STOP
1	7	1	1	8	1	8	1	1

reading:

START	ID	R/W	ACK	ADDR	ACK	STOP
1	7	1	1	8	1	1

START	ID	R/W	ACK	DATA	NACK	STOP
1	7	1	1	8	1	1

chaining:

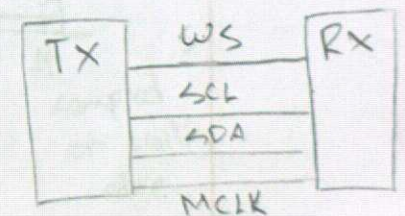
START	ID	R/W	ACK	DATA	ACK	DATA	ACK	DATA	ACK	...
1	7	1	1	8	1	8	1	8	1	

I2S : inter-IC sound, point-to-point

- half-duplex, but supports full duplex if you add 2 signals
- no arbitration since it's point-to-point
- MSB first, WS alignment varies based on modes

WS standard
WS Right Aligned
WS Left Aligned

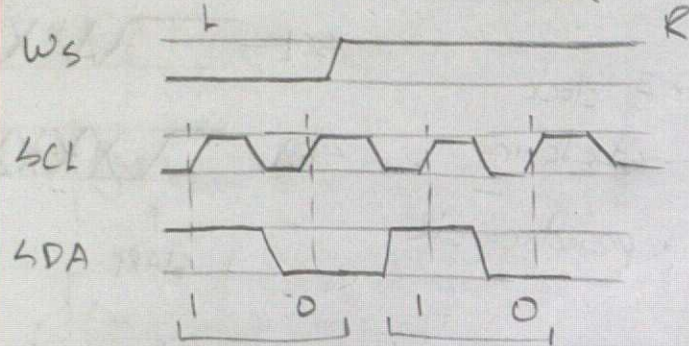
WS Pulse Mode A
WS Pulse Mode B



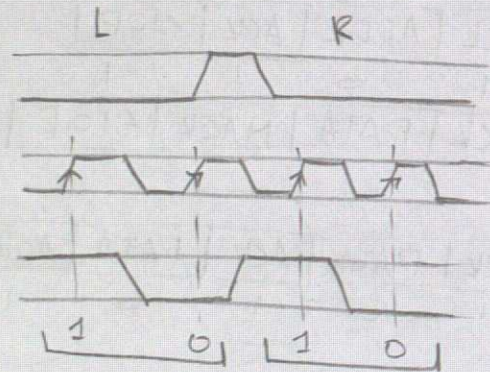
$$\text{SCL frequency} = \text{sound sampling rate} \times \text{bits per channel} \times \text{number of channels}$$

MCLK : 256x or 512x sampling rate

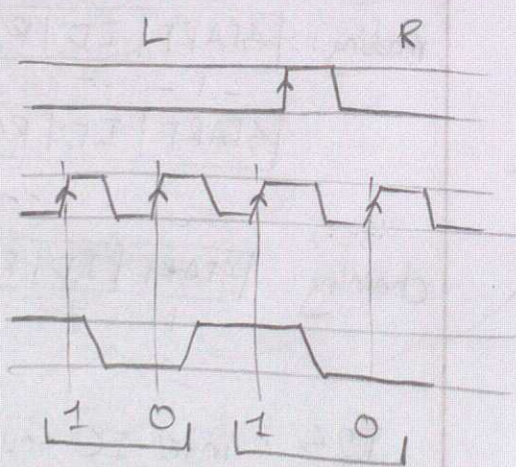
WS standard



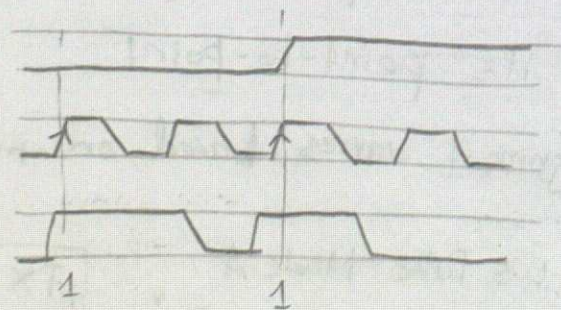
WS Pulse Mode A



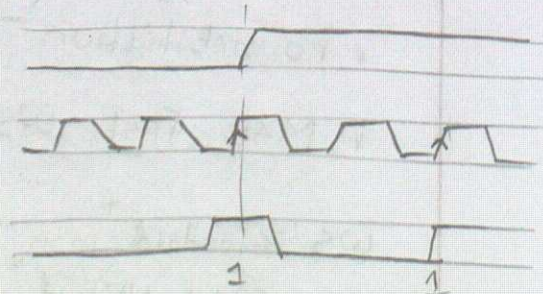
WS Pulse Mode B



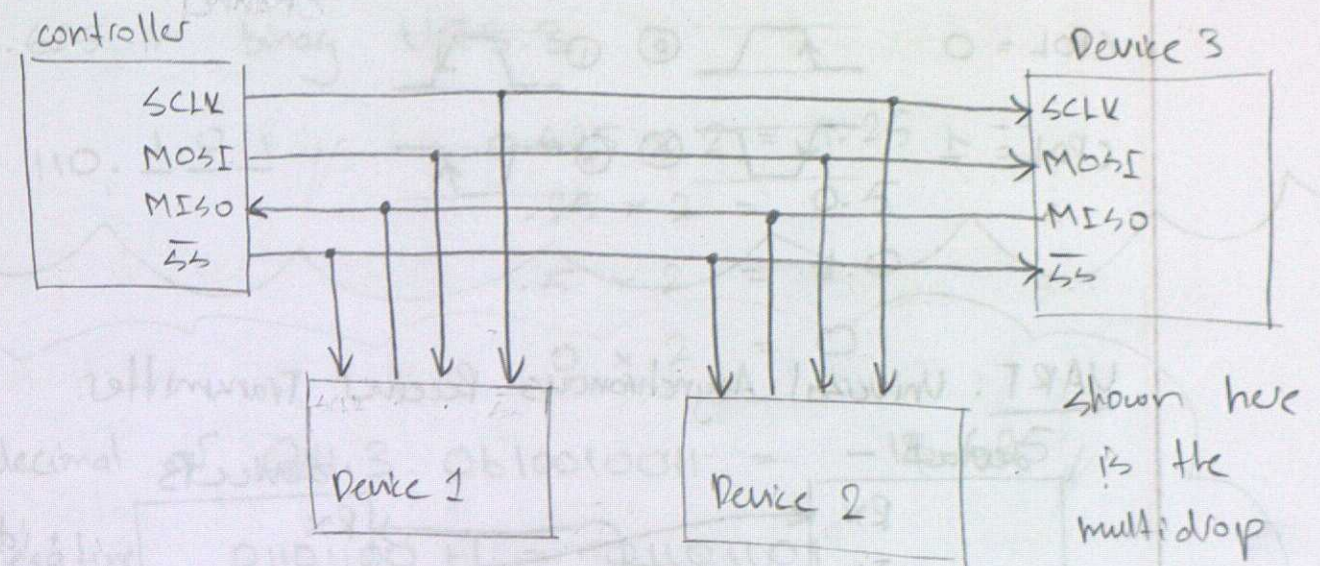
SW Left aligned



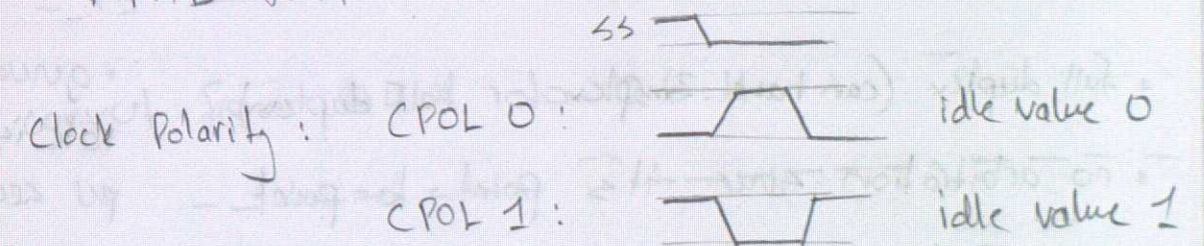
SW Right aligned



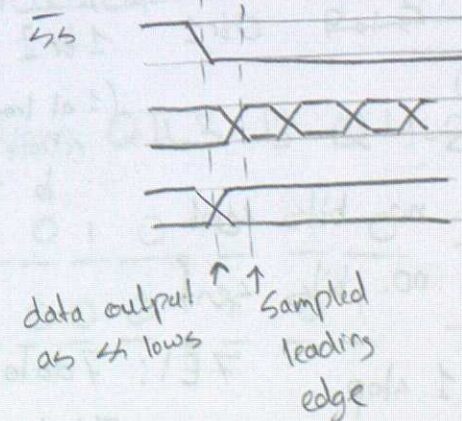
SPI: Serial peripheral interface



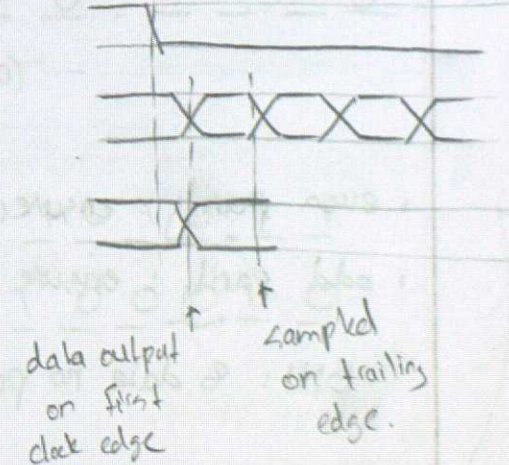
- Full duplex
- no arbitration (once controller selects, only that dev can talk)
- MSB first



Clock Phase: CPHA = 0



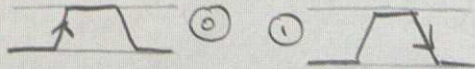
CPHA = 1



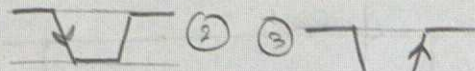
CPHA = 0

CPHA = 1

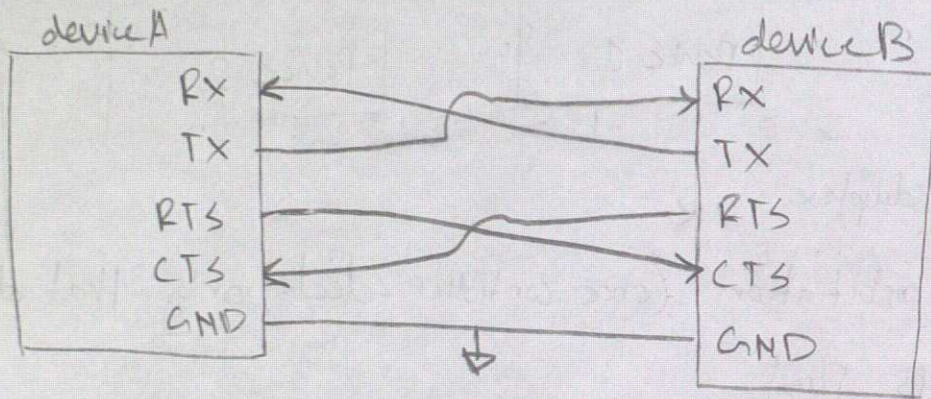
CPOL = 0



CPOL = 1



UART: Universal Asynchronous Receiver Transmitter.



* idle state is HIGH.

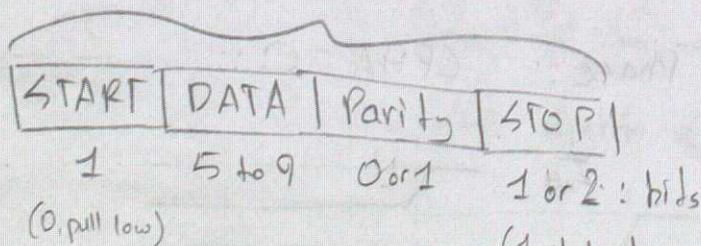
- * full duplex (can have simplex or half duplex)
- * no arbitration since it's point-to-point
- * LSB first

* generally, baud rate is symbols per second.

* UART, each symbol is 1 bit, so baud rate = bits per second.

UART Frame

Standard format:



- * even parity: ensure even no. bits sent
- * odd parity: ensure odd no. bits sent

8N1: 8 data, no parity, 1 stop.

7E1: 7 data, even parity, 1 stop

7O2: 7 data, odd parity, 2 stop.

M14 DMA - DCME

- DMA (general purp. data movement), increase ^{energy} efficiency
 - fly-by DMA, Flow-through DMA
 - which controller, which channel, which trigger?
- DCME (reading images from camera into memory)
 - on I2C-based SCCB

M15 wireless

- low f (long λ): better penetration and range
- high f (short λ): higher bandwidth
- Industrial, Scientific, Medical (ISM) bands unlicensed
- Link budget: must ensure $RX P_{wr} > RX \text{ sensitivity}$

$$RX P_{wr} = TX P_{wr} + TX \text{ gain} + RX \text{ gain} - FSL$$

- Free-space Path Loss (FSPL), d dist., f freq., c speed light

$$FSPL = \left(\frac{4\pi df}{c} \right)^2$$

$$FSPL_{dB} = 20 \log_{10} d + 20 \log_{10} f$$

$$+ 20 \log_{10} \frac{4\pi}{c}$$

if d in km
and f in MHz $\rightarrow$ equals
+32.45 dB

M16 Clock, Power, Energy

- PLL's to generate clocks
- $P = \frac{E}{\Delta t} \rightarrow E = P \cdot \Delta t$ (we care more about E)
- $P = V \cdot i$, $P = P_{static} + P_{dynamic}$
 - $\rightarrow \alpha \cdot (V_{dd})^2 \cdot C_L \cdot f$
 - hardware based (cont ctrl) $\rightarrow$ can ctrl
- current = $244 \mu A / MHz$ @ max voltage $\leftarrow$ includes α and C_L (can $\times$ by V_{dd} and f to get power)
 - $P \propto f$
- P increases exponentially with voltage.
- higher $V \rightarrow$ higher f (since C can be charged faster)
- V determines F_{max} only
- control f and V_{dd} to reduce power $\rightarrow$ low power modes
- 1 Joule = 1 Watt-second
- (energy in Joule) = (energy in watt-hours) $\times 3600$
- (energy in watt-hour) = (current in amp-hour) $\times$ voltage

M17 Memory

- volatile memory (aka RAM)
 - SRAM: 2 inverters in loop + 2 access FETs
 - DRAM: cap with access FET, destructive reads, periodic refresh required
 - DRAM 20x denser but 10x slower than SRAM
- non-volatile memory (aka ROM)
 - PROM, masked ROM
 - Flash memory (Floating gate FET) (NAND Flash
NOR Flash)

M18 Embedded CPU

- ISAs
- single-instruction multiple data (SIMD)
- inline assembly

M19 - FSMs

current state inputs next state outputs

M20 Pipelining

- Latency: time b/n input arriving and corresponding output being produced

$$\text{Latency} = \sum \text{time of all stages}$$

- Throughput: total ~~x~~ inputs that can be processed per unit time

$$TP_{\text{serial}} = \frac{1}{\sum \text{time for all stages}} \quad TP_{\text{fp}} = \frac{1}{\max \{ \text{time for single stage} \}}$$

- Bandwidth: amount of data xferred per unit time

$$BW = TP \times (\text{bits per item})$$

- hazard (structural: multiple stages need same resource
data: consumer using data producer hasn't finished
control: $\exists$ branches leading to diff. behaviour
(must account for longest latency))